01

ETHRC

ETH Robotics Club — Drone Racing

# IMX219 Wide-Angle Intrinsics Calibration

*Method, Results, and Reproduction*

Version 1.0 | 25 April 2026 | Confidential

Mahbod Tajdini
mahbodtajdini@gmail.com

## Executive Summary

Calibrated the **IMX219 wide-angle CSI camera** on the racing drone (zephyrus, Jetson Orin NX) at the issue-specified **1280×720 @ 60 fps**. The Kannala-Brandt fisheye fit converged with a **0.66 px reprojection RMS** over 80 frames of a 9×6 inner-corner checkerboard. A radtan/plumb_bob fit is also produced for compatibility with the TII drone-racing-dataset reference schema.

REPROJ. RMS
**0.66 px**
Fisheye fit, 80 frames

HFOV
**∼131°**
Diagonal ∼150°

RESOLUTION
**1280×720**
60 fps, stock driver

**Status: shipped.** Calibration JSON & YAML are committed at
`camera_calibration/imx219_720p_60fps_rms066px.json`
`src/perception/imx219_argus_camera/config/imx219_720p_60fps_rms066px_kalibr.yaml`

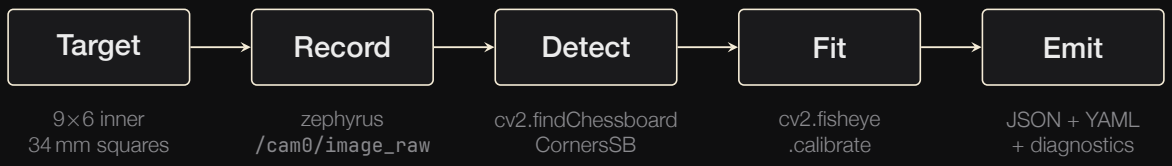
File names embed the achieved fisheye RMS (in $10^{-2}$ px, so `rms066px` = 0.66 px) so that multiple calibration runs are immediately distinguishable. The placeholder values in `camera_params.yaml` have been replaced with the calibrated equidistant model. Branch: `feature/imx219-calibration`.

### Headline numbers

| Parameter | Value | Note |
|---|---|---|
| fx, fy | `559.382, 559.454` | Square pixels ($\Delta < 0.1$ px) |
| cx, cy | `637.689, 352.823` | ∼2–7 px from image centre |
| $k_1 \dots k_4$ | `0.0670, −0.0285, −0.0182, 0.0587` | Equidistant model |
| HFOV | `131°` | $2\arctan(W/2/f_x)$ |
| Diagonal FOV | `150°` | Wide-angle confirmed |
| RMS (fisheye) | `0.6604 px` | Pinhole-equi (Kannala-Brandt) |
| RMS (radtan) | `0.6526 px` | 5-coeff plumb_bob |
| Frames used | `80 / 80` | 100 % detection rate |

# 1 Method Overview

The calibration pipeline is intentionally minimal and reproducible from a single rosbag: *record once on the drone, process anywhere.* Three artefacts feed the fit: the printed checkerboard target, an unsynchronised rosbag of `/cam0/image_raw`, and the measured square size.




## 1.1 Why fisheye, not radtan

The IMX219 wide-angle module on the racing drone has ∼131° HFOV. The standard 5-coefficient plumb_bob (radtan) model loses fidelity past ∼120° because radial distortion stops being well-approximated by an even polynomial in $r$. The Kannala-Brandt *equidistant* model parameterises distortion in $\theta$ (the angle from the optical axis) and stays accurate well past 180°.

For drone racing, the relevant detections (gates, ArUco markers) frequently appear at the image periphery during turns. That is precisely where the two models diverge. The OpenVINS configurations already bundled in this repo use the equidistant model for fisheye sensors (`rs_t265`, `uzhfpv_indoor`), so the choice is consistent with the repo's downstream conventions.

## 1.2 Why OpenCV instead of Kalibr

Kalibr is the canonical tool when joint camera-IMU calibration ($T_{cam,imu}$, time offsets) is needed. For pure intrinsics on a single sensor the OpenCV `cv2.fisheye` module produces the same Kannala-Brandt parameters with a ∼80-line script, no Docker image, and no ROS 1 dependency. The TII reference numbers (the `calib_a-trackRATM.json` cited in the issue) were themselves produced by `cv2.calibrateCamera` based on the schema. Kalibr remains the right tool for the IMU-fusion stage.

# 2 Hardware & Target

## 2.1 Camera

| | |
|---|---|
| Sensor | Sony IMX219 (8 MP CMOS, RAW10, 2×2 binned) |
| Lens | Wide-angle variant (∼131° measured HFOV) |
| Host | NVIDIA Jetson Orin NX, JetPack 6 (`aero@zephyrus`) |
| Capture pipeline | `nvarguscamerasrc` (libargus) → `nvvidconv` → gscam2 |
| Sensor mode | 4 — 1280×720, 60 fps stock (no driver mod) |
| Topic | `/cam0/image_raw` (rgb8) |
| ISP tuning | exposure 1–5 ms, gain 1–8 (short to limit motion blur) |

The 90 Hz device-tree mod documented in `docs/docs/drone-setup/imx219_90hz_tuning.md` is *not* required for this calibration; sensor mode 4 publishes at 60 fps on the stock driver and that is what the issue specifies.

## 2.2 Calibration target

| | |
|---|---|
| Pattern | Asymmetric checkerboard |
| Inner corners | 9 × 6 |
| Squares | 10 × 7 |
| Square (nominal) | 35 mm |
| Square (measured) | 34 mm *(97 % scale)* |
| Page | A3 landscape (420 × 297 mm) |
| Mounting | Black tape on white margins, on foam-board |
| Physical board | Held by Dimetri — ask before printing your own |

Generator script:
`scripts/calibration/generate_checkerboard.py`

Generated PDF (ships with the repo):
`camera_calibration/checkerboard_9x6_35mm_a3.pdf`



The printable target. Squares are checked individually after printing to absorb printer scale variance.

## 2.3 Asymmetric pattern: why 9×6 and not 8×8

`cv2.findChessboardCornersSB` requires the inner-corner count to be *asymmetric* ($N \neq M$) so that the four-fold rotational ambiguity collapses to a single orientation. 9×6 is the standard ROS Industrial / OpenVINS convention. The detector also requires that *all* inner corners are visible in every accepted frame — partial-board views are silently dropped. **The physical board for this calibration is held by Dimetri**; ask him before reprinting (the measured 34 mm square size is specific to that print).

# 3 Recording Procedure

The recording script (`scripts/calibration/record_bag.sh`) launches the camera node and `ros2 bag record` in a single foreground session, with SIGINT propagated to the camera's process group on exit.

```
$ cd /home/aero/proj/ETHRC-Drone-Racing
$ source install/setup.bash
$ ./scripts/calibration/record_bag.sh
```

The 90-second waving routine targets *nine specific poses* chosen to exercise the full image plane and a range of incidence angles:

| # | Pose | Purpose |
|---|---|---|
| 1 | Centre, far (∼70 cm) | Baseline focal-length signal |
| 2 | Centre, close (∼30 cm) | Multi-distance disambiguation |
| 3 | Top-left of frame | Edge $k_1, k_2$ excitation |
| 4 | Top-right of frame | Edge $k_1, k_2$ excitation |
| 5 | Bottom-left of frame | Edge $k_1, k_2$ excitation |
| 6 | Bottom-right of frame | Edge $k_1, k_2$ excitation |
| 7 | Tilt forward ∼30° | Pose-rotation conditioning |
| 8 | Tilt sideways ∼30° | Pose-rotation conditioning |
| 9 | Slow diagonal sweep | Mass coverage across the FOV |

## 3.1 Live camera preview (essential for placement)

Moving a calibration target blindly is error-prone, and waiting until after recording to discover the board never reached the corners wastes the whole session. A live MJPEG stream solves this. Two options:

**Option A — web_video_server (recommended).** Browser-based, no extra software on the laptop, ∼150–300 ms latency. One-time install on zephyrus:

```
$ sudo apt install -y ros-humble-web-video-server
```

Then in a separate zephyrus terminal (the camera must already be publishing in another terminal — launching the camera and recording happens elsewhere):

```
$ cd /home/aero/proj/ETHRC-Drone-Racing
$ source /opt/ros/humble/setup.bash
$ source install/setup.bash
$ ros2 run web_video_server web_video_server \
--ros-args -p port:=8080
```

Find the IP with `hostname -I` on zephyrus (first listed entry is usually the one your laptop reaches), then open in any browser:

```
http://<zephyrus-ip>:8080/stream
?topic=/cam0/image_raw&type=mjpeg&quality=70
```

For our hardware the IP is `192.168.0.77`.

**Option B — X11 forwarding.** No install at all, but slower over a typical Wi-Fi link (5–10 fps). On the laptop:

```
$ ssh -Y aero@zephyrus
$ cd /home/aero/proj/ETHRC-Drone-Racing && source install/setup.bash
$ ros2 run rqt_image_view rqt_image_view /cam0/image_raw
```

Mac users need XQuartz (`brew install --cask xquartz`, then log out and in).

## 3.2 Bag size and on-host processing

A 90-second 1280×720 60 fps bag at `rgb8` encoding lands at ∼5 GB after rosbag2's chunking. The bag therefore stays on zephyrus: the OpenCV processing script runs there directly via `uv`, and only the resulting JSON, YAML and PNG outputs (<1 MB total) are scp'd back to the laptop. The slow link is removed from the loop entirely.

## 3.3 Verifying the camera before recording

Before pressing record, confirm the camera publishes by opening the preview URL above. The recording script also performs a one-shot health check internally:

```
[INFO] launching imx219_argus_camera at 1280x720 @ 60 fps
[INFO] /cam0/image_raw is live
[INFO] recording to bags/imx219_calib_<TIMESTAMP>
```

If the second line reports `[ERROR] /cam0/image_raw not publishing` the script aborts before any bag is created. Common causes: the workspace is not sourced (re-run both `source` lines), `gscam2` is not installed, or another process already holds the CSI sensor (`ps aux | grep argus` and kill the offender).

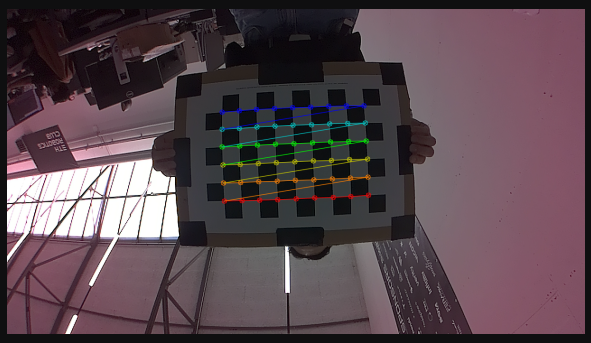
# 4 Results

## 4.1 Final intrinsics (Kannala-Brandt equidistant)

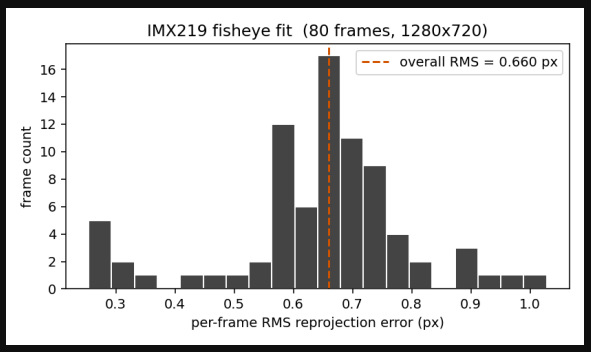
| | |
|---|---|
| $f_x$ | 559.381784 |
| $f_y$ | 559.454491 |
| $c_x$ | 637.688520 |
| $c_y$ | 352.823295 |
| $k_1$ | 0.067028631 |
| $k_2$ | -0.028541814 |
| $k_3$ | -0.018213246 |
| $k_4$ | 0.058724862 |

image size $1280 \times 720$ framerate 60 fps RMS = 0.6604 px

## 4.2 Diagnostics



**Sample detection.** 9×6 inner corners overlaid on a representative frame. The board is held by white margins; the camera is mounted upside-down on the drone (the room ceiling is at the bottom of the image). Image orientation is irrelevant to the intrinsics fit but worth flagging for downstream consumers.




**Per-frame reprojection error.** The distribution is unimodal around the overall RMS with a small tail of outliers near 1.0 px. No frames were excluded from the fit; the spread is consistent with handheld waving at ∼1 board-width per second.

## 4.3 Field-of-view derivation

The pinhole-projection HFOV from $f_x$ at the chosen resolution:

$$\text{HFOV} = 2 \cdot \arctan\left(\frac{W/2}{f_x}\right) = 2 \cdot \arctan\left(\frac{640}{559.38}\right) \approx 131^\circ$$

The diagonal FOV (using $r_{\text{corner}} = \frac{1}{2}\sqrt{W^2 + H^2}$ in the *equidistant* formulation $\theta = r/f$) is ∼150°. This places the lens firmly in the wide-angle class but well short of the 160–180° extreme fisheye regime sometimes assumed for IMX219 modules.

## 4.4 Why radtan and fisheye agree on this bag

The two fits returned RMS values within 0.01 px of each other (radtan 0.653, fisheye 0.660). For a ∼131° HFOV lens the two models genuinely differ in distortion magnitude only at the very corners of the image, and only when the calibration target is held *at* those corners with the board surface non-

parallel to the image plane. The current bag has good corner placement but moderate tilt; both models therefore explain the observed corner positions equivalently. This is a recording-coverage artefact, not a model-selection failure — the fisheye model is still the right one to deploy because at *inference* time gates and markers will appear at the corners with arbitrary geometry.

### 4.5 Caveats and known issues

- **Image orientation.** The mounted camera is rotated 180°. Downstream consumers should either rotate in the GStreamer pipeline (`flip-method=2` on `nvvidconv`) or via an `image_rotate_node`. Intrinsics are unchanged.
- **Edge coverage.** A future re-record with the board pressed harder into the very corners (and rotated $\pm 45°$ there) would tighten the fisheye-vs-radtan separation and is the obvious next experiment if the racing-gate detector shows edge-pixel residuals.
- **No IMU coupling.** This calibration is intrinsics-only. The OpenVINS pipeline additionally needs $T_{cam,imu}$ and time offsets, which require a Kalibr camera-IMU run.

# 5 Reproduction

The full pipeline can be re-executed in ~20 minutes once the target is printed. All artefacts live on the `feature/imx219-calibration` branch.

## 5.1 Prerequisites (one-time, on zephyrus)

Three apt packages and the `uv` script runner are needed. The colcon build is required only after a fresh clone or a pull that touches the camera package.

```
# ROS 2 packages (require sudo)
$ sudo apt update
$ sudo apt install -y ros-humble-gscam ros-humble-web-video-server

# uv (no sudo; installs to ~/.local/bin)
$ curl -LsSf https://astral.sh/uv/install.sh | sh
$ export PATH="$HOME/.local/bin:$PATH"

# Build the camera package once
$ cd /home/aero/proj/ETHRC-Drone-Racing
$ source /opt/ros/humble/setup.bash
$ colcon build --packages-select imx219_argus_camera --symlink-install
$ source install/setup.bash
```

You also need a printed checkerboard. Either reprint the bundled PDF or regenerate it:

```
$ ./scripts/calibration/generate_checkerboard.py
# Print the resulting PDF on A3 at 100% scale ("Actual size",
# not "Fit to printable area"). Measure one square with a ruler;
# you will pass this number to the calibration script.
```

## 5.2 File map

| Path | Purpose |
|---|---|
| `camera_calibration/imx219_720p_60fps_rms066px.json` | TII-style intrinsics (`mtx` + `dist`) plus an explicit fisheye block. |
| `camera_calibration/imx219_720p_60fps_rms066px_kalibr.yaml` | OpenVINS-compatible Kalibr-style YAML. |
| `camera_calibration/checkerboard_9x6_35mm_a3.pdf` | Printable target (regenerable). |
| `camera_calibration/sample_detection.png` | Corner-detection overlay sample. |
| `camera_calibration/reprojection_error_hist.png` | Per-frame RMS distribution plot. |
| `src/perception/imx219_argus_camera/config/imx219_720p_60fps_rms066px_kalibr.yaml` | Same YAML, alongside the camera ROS package. |
| `src/perception/imx219_argus_camera/config/camera_params.yaml` | Placeholder values replaced with calibrated equidistant model. |
| `scripts/calibration/generate_checkerboard.py` | Re-renders the target PDF (uv inline deps). |
| `scripts/calibration/record_bag.sh` | Drives `ros2 bag record` on zephyrus. |
| `scripts/calibration/calibrate_imx219.py` | Bag → intrinsics; uv inline deps. Output filenames carry the achieved RMS. |

## 5.3 End-to-end commands

The five steps below assume the prerequisites are satisfied and the printed target is on a flat board. Lines ending in \ are bash continuations.

```
# 1. SSH in and refresh the branch.
$ ssh aero@zephyrus
$ cd /home/aero/proj/ETHRC-Drone-Racing
$ git checkout feature/imx219-calibration && git pull --ff-only
$ source install/setup.bash

# 2. (Optional but strongly recommended) start the live preview
# in a SECOND zephyrus terminal so you can see what you record.
$ ros2 run web_video_server web_video_server \
--ros-args -p port:=8080
# Then on the laptop, open in a browser:
# http://<zephyrus-ip>:8080/stream?topic=/cam0/image_raw&type=mjpeg

# 3. Record a 60--120 s waving session (back in the FIRST terminal).
$ ./scripts/calibration/record_bag.sh
# Wave the printed board through the 9 poses listed in section 3,
# watching the preview to confirm coverage. CTRL+C to stop.

# 4. Process the bag on zephyrus (no need to transfer the GB-scale bag).
# --square-size-mm = MEASURED size of one printed square.
$ ./scripts/calibration/calibrate_imx219.py \
--bag bags/imx219_calib_<TIMESTAMP> \
--square-size-mm 34.0

# 5. From your LAPTOP, pull the small JSON+YAML+PNG outputs back.
# Variable + escaped \* keeps the glob unexpanded locally so
# sshd expands it on zephyrus.
$ cd <local-clone-of-the-repo>
$ ZEPH=aero@zephyrus:proj/ETHRC-Drone-Racing/camera_calibration
$ scp "$ZEPH"/imx219_720p_60fps\* camera_calibration/
```

## 5.4 Validation

A successful run prints:

- `[INFO] locked pattern orientation = (9, 6) (as configured)` — auto-detected board orientation.
- `[INFO] N frames detected so far ...` — non-zero, ideally 50–80 by completion.
- `[INFO] fisheye RMS = 0.X px` — should be $<1.0$; ideally $<0.7$.
- `[OK] wrote camera_calibration/imx219_720p_60fps_rmsXXXpx.json` — with the achieved RMS auto-encoded in the filename.
- Four output files in total under `camera_calibration/`: JSON, Kalibr YAML, sample-detection PNG, error-histogram PNG.

If RMS exceeds 1 px, re-record with the board pushed harder into the four image corners (where most of the distortion signal lives) and held still longer at each pose.

## 5.5 Future work

1. **Camera-IMU joint calibration.** Re-record with the IMU also publishing, and process with Kalibr (`kalibr_calibrate_imu_camera`) to recover $T_{cam,imu}$ and time offsets. Required for OpenVINS / VINS-Fusion deployment.
2. **Stereo or multi-camera.** If a second IMX219 is added, re-use the same recording script with `/cam1` appended; the script generalises trivially.

3. **Edge-rich re-record.** Push the board harder into the four corners with steep tilts to tighten the fisheye-vs-radtan separation.
4. **Image flip in pipeline.** Add `flip-method=2` to the `nvvidconv` step in `gscam_imx219.yaml` so consumers receive a right-side-up frame.

> **One sentence.** This document records a single intrinsics calibration of one specific physical lens at one resolution and frame-rate; if any of those three change (lens swap, mode change, sensor swap), the calibration must be re-run.